Memory-Aware Agent Serving: Phase 1 Report

Zhe Kan CSE 291A, Fall 2026, Option 1

1 Question and setup

Agent tasks issue chains of dependent model requests whose context grows at every step. On a memory-constrained machine, admitting too much can exhaust memory, while admitting too little leaves capacity idle during tool delays. Phase 1 asks how concurrency, context growth and the memory budget affect completed work, whether retained context matters, what the dominant bottleneck is, and which mechanism Phase 2 should build. Prior systems schedule at iteration level [1], page and preempt KV cache [2], bound prompt-processing stalls [3], share prefixes [4] or schedule whole agent programs [5]. We instead study admission by an external proxy in front of an unmodified runtime.

System. An asyncio proxy runs a pluggable admission policy in front of llama-server b11606 [6], serving Llama-3.2-3B-Instruct Q4_K_M [7, 8] on an RTX 2000 Ada (16 GB). The **memory budget** is the token capacity of the server's shared KV-cache pool. A pilot found that the server admits by slot count only, never pauses a running sequence, and does not report KV occupancy. When the pool overflows and no idle context is left to delete, it fails *every* running sequence at once; we call this an **overflow abort** (Section A.1).

Workload and baselines. Synthetic agent tasks come from 8 task classes: 500 or 4,000 starting tokens, 50 or 400 output tokens per step, and 2–3 or 8–12 steps. Each step adds a 200-token tool result and a log-normal tool delay (1 s median). Prompts are token arrays, so prefixes extend exactly, and output lengths are fixed. Arrivals are open-loop Poisson [9]. The parameters were checked against 500 SWE-agent trajectories [10, 11]. **Task-level FIFO** holds one of N slots per agent task for its whole life; **request-level FIFO** admits each model request with at most N in flight. $N \in \{1, \ldots, 16\}$ was tuned at 0.1 agent tasks/s by a fixed rule, with resident, offload or no prompt caching.

Protocol and metrics. Each run has a 60 s warm-up, an 8-minute arrival window and a drain, on a fresh server, with 5 seeds per configuration and 95% CIs. Metrics follow the agent tasks that *arrive* in the window through the drain, and rates divide by the 8-minute window. **Goodput** counts agent tasks completed within their SLO: 3× their unloaded completion time on one fixed reference setup. A **failed task** returned an error or was unfinished at its **hard timeout**, 10× its unloaded time measured from arrival, covering queueing, requests and tool delays. Latency percentiles cover completed tasks only, so a configuration that loses long tasks can look faster. We analysed 200 runs on two RunPod machines. They were pooled only after an equivalence check: median per-task time ratio 1.039, and every headline metric within the first machine's CI (Section A.5).

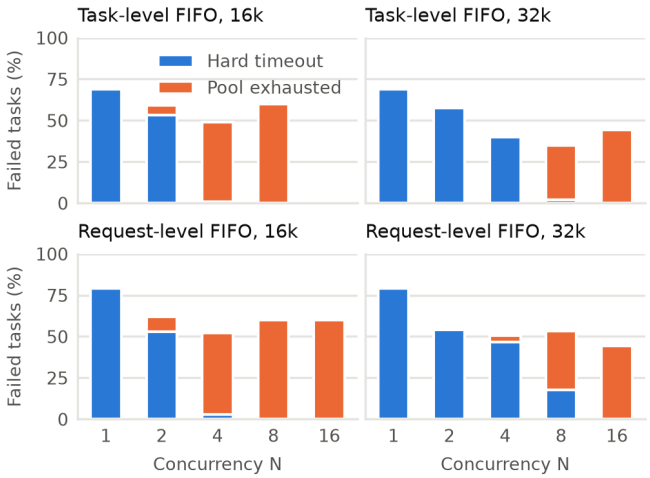


Figure 1: Failed agent tasks by cause against N at 0.1 agent tasks/s, resident caching.

Configuration (0.1/s)	Goodput/min	Timeout %	Pool %	p50 s
Task, N=1, 16k	0.28 ± 0.37	69	0	359
Task, N=4, 16k*	2.73 ± 0.53	1	48	20
Task, N=8, 16k	2.58 ± 0.61	0	60	16
Request, N=8, 16k*	2.58 ± 0.61	0	60	17
Task, N=4, 32k	1.12 ± 0.51	40	0	154
Task, N=16, 32k*	3.12 ± 1.18	0	44	27

Table 1: Selected results (*: best N). Full grid: Section G.1.

2 Results

Concurrency has two failure modes. Small N queues tasks until their hard timeout; large N overflows the pool (Figure 1). **Even the best N fails about half of all agent tasks**, and neighbouring settings' CIs overlap, so tuning reaches a plateau rather than a peak (Table 1). Below the point where N binds, request-level FIFO is worse, because re-queueing every step loses retained context (75 against 91% of prompt tokens cached at N=1).

Overflow is sudden and collective. At task-level N=4 on 16k there were 6.4 ± 1.4 aborts per run, each failing 3.8 of the 4 running requests (Figure 2). Yet the mean admitted footprint was only 52% of the budget (21–52% across configurations), so pressure is episodic rather than sustained. 63% of recorded output tokens belonged to agent tasks that eventually failed. At aborts no agent task was idle, consistent with the runtime deleting idle context first; four runs were checked directly (Section B.2).

More memory alone does not fix it. Doubling the budget at N=4 removes aborts, but goodput falls from 2.73 to 1.12 as failures turn into hard timeouts (Table 1). At 0.1 agent tasks/s the mix offers about 141 output tokens/s, close to the most output observed per second of arrival window (about 150, drain included, so not a steady-state capacity). This suggests that com-

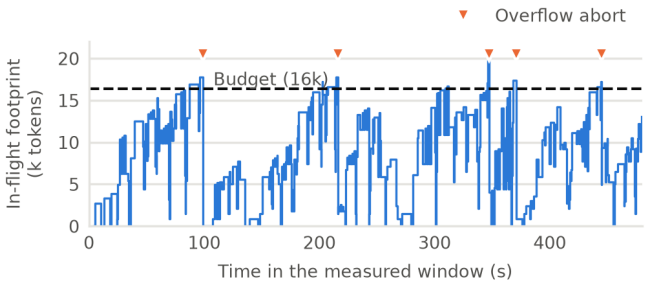


Figure 2: Projected footprint of running requests over one run (task-level, N=4, 16k, 0.1/s): it builds up, crosses the budget and collapses at each overflow abort.

pute pressure may also limit gains at this load (Section B.3).

Load, fairness and caching. At best N, the share of agent tasks within SLO falls from 84 to 70 to 46% (16k) and from 100 to 78 to 48% (32k) as the rate rises from 0.025 to 0.05 to 0.1/s. On 16k every failure at the lower rates is pool exhaustion, so memory is the binding limit there (Section B.4). Long-output, many-step agent tasks fail 99–100% at N=4 on 16k, although every class fits the budget alone. Tuned goodput is therefore partly achieved by losing the longest work (Section B.5). Turning caching off cuts goodput 4.8× (task-level) and 5.9× (request-level). Offloading idle context to host RAM showed no clear benefit under the tested conditions: five seeds, one tool delay (Section B.6). VRAM follows the budget (about 113 KB per token), not the load. Policy overhead (20–30 µs per decision) and cold start are negligible (Section B.7).

3 Bottleneck and Phase 2 direction

Below about 0.1 agent tasks/s, the bottleneck is the combined KV footprint of concurrently running requests, on a runtime that can only fail all of them when they overflow. A slot count is the wrong unit: four short tasks and four long ones differ in footprint by an order of magnitude. At 0.1 agent tasks/s, compute pressure probably also matters, but this is inferred rather than measured. **Phase 2 will build footprint-based admission:** the proxy holds a model request when its known peak footprint (prompt plus output length), added to that of everything in flight, would exceed the budget. The projected footprint was over the budget during 145 of 146 aborts. That is partly by construction, so it shows consistency, not prevention (Section D). Tuning N, a larger budget, offloading and choosing what to evict were weighed and not pursued for now.

Hypotheses. H1: at 0.025–0.05 agent tasks/s it removes almost all aborts and beats the best tuned N on goodput. **H2:** at 0.1 agent tasks/s the gain is smaller if compute limits it, and a sustained-capacity measurement will test this first. **H3:** waiting for room can starve long agent tasks, so results are reported per task class. Evaluation uses held-out seeds and mix, bursty arrivals, both budgets, an ablation and policy overhead.

Limitations. Tool delay was not varied, which could change the offload and eviction conclusions. Footprints are the proxy's projections, not the runtime's occupancy. Runs have 11–51 measured tasks with correlated failures, so CIs are wide. The workload is synthetic, with one model and one GPU type, and the two machines differ by about 4% per task (Section E).

Reproducibility and AI use. The code, data (release `phase1-data`) and scripts that regenerate every table and figure are public at `https://github.com/zkan2002/CSE291A-Project` (Section F). The author made every design and experimental decision, decided which conclusions this report draws and the Phase 2 direction, and checked the claims against the data. AI assistance (Claude, Anthropic) was used to implement most of the code, compute the analyses, generate figures, draft text and propose options and interpretations, which the author accepted, revised or rejected. All data come from the author's own experiments.

References

[1] Gyeong-In Yu et al. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.

[2] Woosuk Kwon et al. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.

[3] Amey Agrawal et al. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.

[4] Lianmin Zheng et al. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[5] M. Luo et al. Agentix: An efficient serving engine for LLM agents as general programs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026.

[6] ggml-org. llama.cpp, release b11606 (commit 1b70350c92563449576bcbbf552994b1d1ed847d). `https://github.com/ggml-org/llama.cpp/releases/tag/b11606`, 2026. Server source at that commit: `https://github.com/ggml-org/llama.cpp/blob/1b70350c92563449576bcbbf552994b1d1ed847d/tools/server/server-context.cpp`.

[7] Meta. Llama-3.2-3B-Instruct model card. `https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct`, 2024.

[8] bartowski. Llama-3.2-3B-Instruct-GGUF, revision 5ab33fa94d1d04e903623ae72c95d1696f09f9e8, file Llama-3.2-3B-Instruct-Q4_K_M.gguf. `https://huggingface.co/bartowski/Llama-3.2-3B-Instruct-GGUF/tree/5ab33fa94d1d04e903623ae72c95d1696f09f9e8`.

[9] Bianca Schroeder, Adam Wierman, and Mor Harchol-Balter. Open versus closed: A cautionary tale. In *3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2006.

[10] John Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[11] Nebius. SWE-agent trajectories dataset. `https://huggingface.co/datasets/nebius/SWE-agent-trajectories`.

A Method

A.1 System and runtime behaviour

The harness has a workload generator, an asyncio proxy that runs a pluggable admission policy, an unmodified `llama-server`, and a recorder. The memory budget is the token capacity of the server's shared KV-cache pool (`-kv-unified`). We control it directly and measure physical memory as an outcome, because the server reserves the pool at launch, so a byte limit would mostly make it fail at startup.

A pilot established the behaviour everything else depends on, at the pinned release. The pilot findings (`docs/pilot/pilot-findings.md`) record the probes and logs; the server source at the pinned commit is cited in [6]:

- Admission counts slots only. It never checks whether a sequence will fit.
- When the pool fills, the server deletes one idle slot's KV and retries, then halves its batch size.
- If that still fails, **every running sequence fails** with HTTP 500 and loses its KV. We call this an **overflow abort**. The server stays up, and queued requests still run.
- A request whose prompt plus output exceeds the pool returns HTTP 200 with shortened output. The harness treats this as a failure.
- The server does not report KV occupancy, so the proxy tracks footprints itself.

A.2 Workload

Workloads are synthetic so that the experiment controls token counts exactly. Each agent task belongs to one of eight task classes: short or long starting context (500 or 4,000 tokens) $\times$ short or long output per step (50 or 400) $\times$ few or many steps (2–3 or 8–12). Each step adds its output plus a 200-token synthetic tool result, so final context ranges from about 750 to about 11,000 tokens. Tool delays are log-normal with a 1 s median. Arrivals are open-loop Poisson, so overload appears as queueing instead of being hidden by clients slowing down [9].

Prompts are token arrays, and each step's prompt is an exact extension of the previous one, so measured prefix reuse reflects the runtime and not tokenizer drift. Output lengths are fixed (`n_predict`, EOS ignored), which means no policy can look better by producing shorter outputs. We checked the parameters against 500 public SWE-agent trajectories [10, 11] (median 16 steps, about 1.9k starting tokens, 79 output tokens per step). These are our sample statistics, computed by `scripts/trajectory_stats.py`; the workload analysis in `docs/workload.md` documents the sampling method. Tool delay has no empirical basis, because the dataset has no timings.

A.3 Baselines and configurations

- **Task-level FIFO:** an agent task holds one of N slots from arrival to completion, tool delays included.
- **Request-level FIFO:** each model request queues on its own with at most N in flight, and a task re-queues after each tool delay.

We tuned each baseline by goodput over $N \in \{1,2,4,8,16\}$ at a rate of 0.1 agent tasks/s, with a fixed selection rule (highest mean goodput, ties to smaller N). We then ran the best N for each baseline and budget across rates. Cache policies were **resident** (context stays in the pool, tasks pinned to slots), **offload** (idle context moves to host RAM, tasks unpinned) and **off**.

A.4 Protocol and metrics

Each sweep run has a 60 s warm-up (recorded but excluded), 8 minutes of measured arrivals and a drain, on a freshly started server. Each configuration uses five seeds; we report means with 95% confidence intervals (Student's t). Nothing is retried silently.

What is counted, and over what period. A run's metrics cover the agent tasks that *arrive* during the 8-minute measured window. Each of those is followed through the drain to its outcome: completion, failure or hard timeout. Throughput and goodput are the number of those tasks that completed, or completed within their SLO, divided by the length of the arrival window (8 minutes). Completions that happen during the drain count; tasks that arrive during warm-up or drain do not. Peak memory, KV occupancy and policy overhead cover the same tasks' whole lifetime, through the drain.

Which tasks each metric includes. Failed tasks count as agent tasks in failure rate and SLO attainment, and their model requests count in queueing time. Completion-latency percentiles (p50, p95, p99), however, are computed over **completed agent tasks only**, because a failed task has no completion time. This matters when reading the tables. A configuration that loses its long agent tasks can show *lower* latency than one that finishes them, because the slowest tasks are missing from the sample. Latency should therefore always be read together with failure rate and the per-class breakdown (Section B.5). Cached-token shares are likewise computed over successful model requests.

Goodput and failure. Goodput, the headline metric, counts agent tasks completed within their **task SLO** per minute. The SLO is 3$\times$ the task's unloaded completion time, measured by running it alone on one fixed reference setup (resident, caching on, 32k), so the yardstick does not vary with the configuration under test. A **failed task** is one in which a model request returned an error,

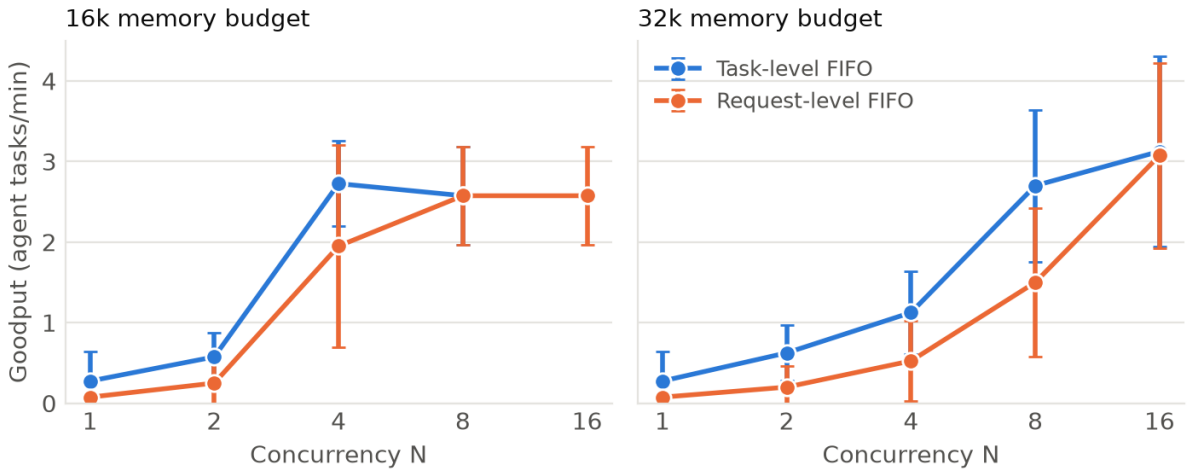


Figure 3: Goodput against N at 0.1 agent tasks/s, resident. Bars are 95% CIs.

Baseline	Budget	N	Mach.	Goodput/min	Throughput /min	Timeout %	Pool exhausted %	p50 s	Queueing s
Task-level	16k	1	A	0.28 ± 0.37	1.98 ± 0.30	69 ± 4	0 ± 0	359 ± 80	50.1 ± 7.4
Task-level	16k	2	A	0.57 ± 0.30	2.62 ± 0.67	53 ± 6	6 ± 4	178 ± 29	26.9 ± 4.2
Task-level	16k	**4**	A	$\mathbf{2.73 \pm 0.53}$	3.27 ± 0.82	1 ± 1	48 ± 9	20 ± 4	1.0 ± 1.2
Task-level	16k	8	A	2.58 ± 0.61	2.58 ± 0.61	0 ± 0	60 ± 8	16 ± 5	0.0 ± 0.0
Request-level	16k	**8**	A	$\mathbf{2.58 \pm 0.61}$	2.58 ± 0.61	0 ± 0	60 ± 8	17 ± 5	0.0 ± 0.0
Task-level	32k	4	A	1.12 ± 0.51	3.85 ± 0.91	40 ± 8	0 ± 0	154 ± 26	15.6 ± 3.1
Task-level	32k	8	A	2.70 ± 0.94	4.15 ± 0.79	2 ± 5	33 ± 5	35 ± 7	1.1 ± 1.8
Task-level	32k	**16**	B	$\mathbf{3.12 \pm 1.18}$	3.60 ± 1.11	0 ± 0	44 ± 12	27 ± 5	0.0 ± 0.0
Request-level	32k	**16**	B	$\mathbf{3.08 \pm 1.15}$	3.60 ± 1.11	0 ± 0	44 ± 12	27 ± 5	0.0 ± 0.0

Table 2: Selected tuning results at 0.1 agent tasks/s, resident (bold: chosen N). The full grid is in Section G.1.

or which was still unfinished at its **hard timeout**: 10× its unloaded completion time, measured from the task's arrival. The hard timeout therefore bounds the whole agent task, including queueing, every model request and every tool delay, not any single model request.

A.5 Environment and the machine change

The model is Meta's Llama-3.2-3B-Instruct [7], using bartowski's Q4_K_M GGUF artifact at revision `5ab33fa9` [8] (SHA256 pinned), on llama.cpp b11606 built from source at commit `1b70350c9` [6], on an RTX 2000 Ada (16 GB) RunPod pod.

The first pod's host failed partway through the sweep, so the experiments span two machines with the same GPU model but different cards, drivers and CPU quotas. We pooled results only after an explicit check. Re-running one configuration with the same seeds on machine B gave a median per-task unloaded-time ratio of 1.039 (p10 1.003, p90 1.050, 279 tasks) against a ±5% tolerance, with all six headline metrics inside machine A's 95% CI. Machine B is therefore consistently about 4% slower per task.

No configuration's seeds span machines, and each machine sets its own SLOs. Points compared across machines are marked † below. Tables mark the machine as A or B.

B Results

B.1 Concurrency has two different failure modes

Small N fails by queueing: tasks wait until they hit their hard timeout. Large N fails by overflow (Figure 4). In between there is no setting that avoids failure: **even the best N fails about half of all agent tasks.** The best N is also a plateau rather than a peak (Figure 3). The confidence intervals of neighbouring settings overlap, so the selected values should be read as "tuning has run out", not as a precise optimum.

Latencies in Table 2 cover completed agent tasks only, so they must be read with the failure columns. Task-level FIFO at $N = 4$ and 16k reports a p50 of 20 s, well below the 65–79 s that a long-output, many-step task takes even when it runs alone (Table 5). Almost none of those tasks completed (Section B.5), so they are simply missing from the latency sample.

Once N no longer binds, the two baselines behave identically, because queueing is zero and the limit is never reached. Below that point request-level FIFO is worse, because re-queueing every step breaks retained context: at $N = 1$ and 16k its cached-token share is $75 \pm 15\%$, against $91 \pm 1\%$ for task-level FIFO.

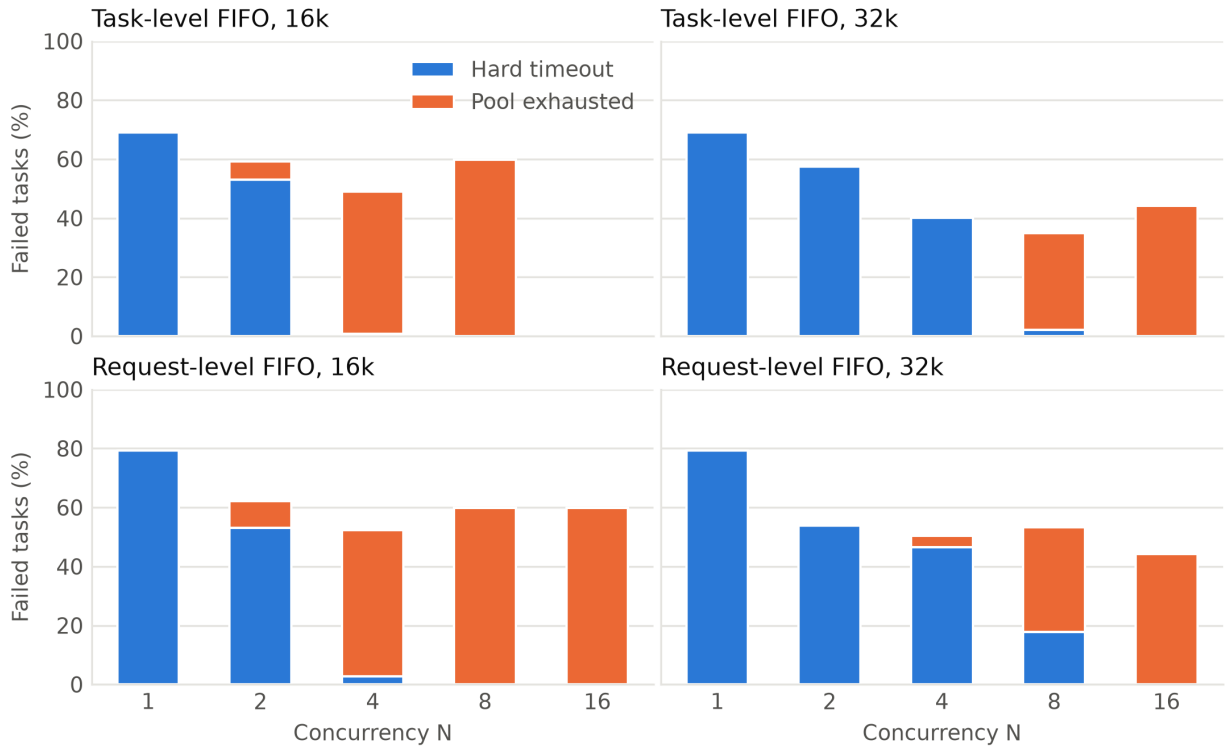


Figure 4: Failed tasks by cause at 0.1 agent tasks/s. $N = 16$ at 16k was run for request-level FIFO only.

Configuration	Mach.	Aborts per run	Failed requests per abort	Footprint at abort (% of budget)	Mean admitted (% of budget)	Recorded output on failed tasks %
Task-level, $N = 4$, 16k, 0.025/s	B	0.8 ± 1.0	2.8	104	21	21 ± 27
Task-level, $N = 4$, 16k, 0.05/s	B	2.2 ± 1.0	3.5	106	35	32 ± 14
Task-level, $N = 4$, 16k, 0.1/s	A	6.4 ± 1.4	3.8	108	52	63 ± 7
Request-level, $N = 8$, 16k, 0.1/s	A	6.2 ± 1.0	4.9	117	42	58 ± 7
Task-level, $N = 16$, 32k, 0.05/s	B	0.4 ± 0.7	8.0	111	35	15 ± 25
Task-level, $N = 16$, 32k, 0.1/s	B	2.4 ± 0.7	9.3	111	43	39 ± 20
Request-level, $N = 16$, 8k, 0.1/s	A	12.8 ± 2.2	2.9	118	35	62 ± 9

Table 3: Overflow events, computed from the request and KV logs. Footprints are the proxy's projections (Section E). Recorded output counts the tokens returned by model requests that completed. A request cut off by an abort returns nothing, so the tokens it had already generated are not counted. Full table with over-budget spells: Sections G.4 and G.5.

B.2 Overflow is a sudden collective failure

Figure 5 shows the pattern in one run, and Table 3 quantifies it. Three things stand out:

- **Failures are correlated.** One abort takes out almost everything running (3.8 of 4 at $N = 4$), so one event costs several agent tasks and all the work they had done. That is why so much of the recorded output belongs to tasks that eventually fail.
- **Pressure is episodic, not sustained.** Mean admitted footprint is only 21–52% of the budget, yet aborts still occur. Capacity exists on average; the failure comes from brief coincidences of large requests. This is the opening for an admission policy.
- **Aborts happen when nothing is idle.** Across the configurations in Table 3 the proxy's estimate of retained context at the moment of an abort is essentially zero, and in the four runs we checked directly (up to four aborts each), request logs confirm every agent task in the system was running. This is consistent with the runtime deleting idle context first, so an abort comes only after that option is used up. Idle-slot deletions are rare in the server logs (about 5 per run at $N = 4$, 16k, 0.1/s).

B.3 More memory alone does not fix it

Doubling the budget at fixed $N = 4$ removes aborts completely, but goodput *falls* from 2.73 ± 0.53 to 1.12 ± 0.51, as failures shift from pool exhaustion (48%) to hard timeouts (40%). Throughput does not fall (3.27 ± 0.82 to 3.85 ± 0.91, CIs overlapping). Our interpretation is that at 16k aborts shed load, leaving compute for the survivors, whereas at 32k all tasks stay and queue until they miss their SLOs. Goodput therefore cannot be read as "higher is healthier" without the failure breakdown.

This suggests a second limit, though Phase 1 did not measure it directly. At 0.1 agent tasks/s the default mix offers about 141 output tokens/s by construction (1,406 output tokens per task on average). The most output we observed at that rate is about 150

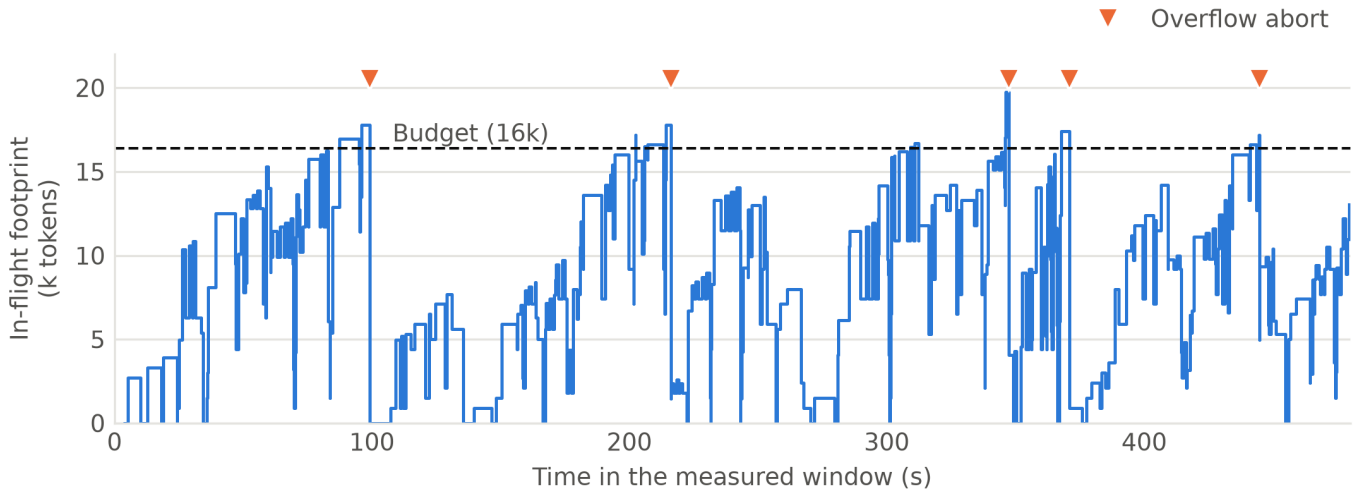


Figure 5: Projected footprint of running requests (prompt plus `n_predict`) over one run: task-level FIFO, $N = 4$, 16k, 0.1 agent tasks/s, seed 1. Footprint builds up, crosses the budget and collapses when the abort fails everything running.

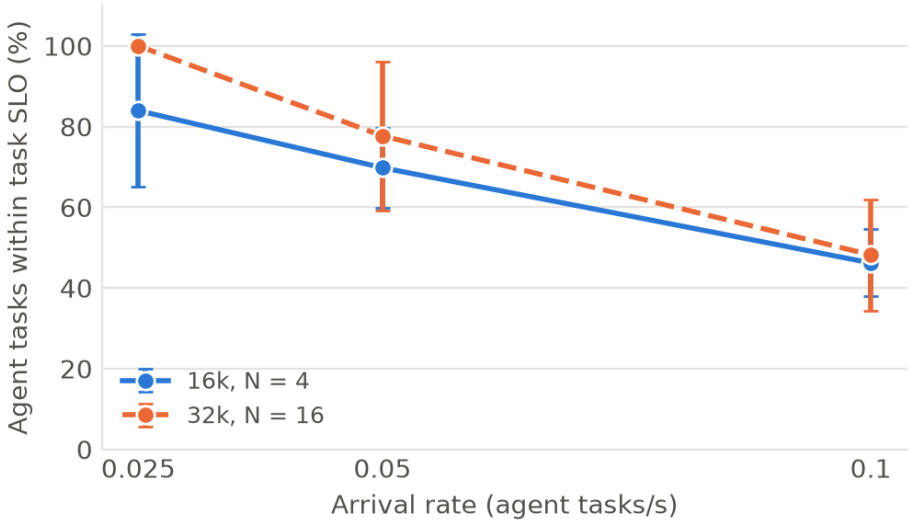


Figure 6: Share of agent tasks within their SLO for task-level FIFO at each budget's best N, all on machine B.

tokens per second of arrival window, among resident configurations (request-level $N = 2$, 32k: 152 ± 26; Section G.6).

That second number is **not a steady-state capacity**. It is all output generated for the measured agent tasks, including output generated during the drain, divided by the length of the arrival window. Still, three observations together suggest that **compute pressure at 0.1 agent tasks/s may limit what any memory policy can gain**:

- offered load is close to the most output observed;
- small N queues until tasks time out;
- $N = 4$ at 32k times out 40% of tasks with no aborts at all.

Confirming this needs a sustained capacity measurement, which Section D plans.

B.4 At lower load, memory is the limit

At 0.025–0.05 agent tasks/s, output per second of arrival window is 34–74 tokens, half or less of the most observed at 0.1 agent tasks/s, yet tasks still fail (Figure 6 and Table 4). On the 16k budget those failures are all pool exhaustion, which points to memory as the binding constraint at these rates. Even at 0.025 tasks/s, about one task arrival every 40 s, 16% of agent tasks fail on a 16k budget. Variability is high at these rates because each run has few arrivals and abort failures are correlated.

The cliff is steepest with a very small budget: request-level FIFO at $N = 16$ and 8k fails $72 \pm 7\%$ of agent tasks, nearly all from pool exhaustion. Only one of the eight task classes (12% of agent tasks) cannot fit in 8k alone, yet the seven that fit also fail, at 14–96% (Section G.8), so most of this cliff comes from tasks colliding rather than from tasks that could never fit.

Baseline	Budget	N	Rate/s	Mach.	Goodput/min	Within SLO %	Failed %	p95 s
Task-level	16k	4	0.025	B	1.12 ± 0.27	84 ± 19	16 ± 19	77 ± 13
Task-level	16k	4	0.05	B	2.20 ± 0.65	70 ± 10	30 ± 10	88 ± 16
Task-level	16k	4	0.1	B	2.95 ± 0.63	46 ± 8	48 ± 10	51 ± 41
Request-level	16k	8	0.025	B	1.10 ± 0.28	82 ± 22	18 ± 22	75 ± 15
Request-level	16k	8	0.05	B	2.12 ± 0.57	68 ± 11	32 ± 11	79 ± 29
Request-level	16k	8	0.1 †	A	2.58 ± 0.61	40 ± 8	60 ± 8	41 ± 21
Task-level	32k	16	0.025	B	1.40 ± 0.60	100 ± 0	0 ± 0	93 ± 27
Task-level	32k	16	0.05	B	2.42 ± 0.66	78 ± 18	13 ± 22	134 ± 42
Task-level	32k	16	0.1	B	3.12 ± 1.18	48 ± 14	44 ± 12	97 ± 54

Table 4: Arrival-rate ladder at best N. † measured on a different machine from the rest of its series. Request-level FIFO at 32k is in Section G.2.

Task class (output, steps, starting context)	Median unloaded s	Failed % $N = 4$, 16k (A)	Failed % $N = 16$, 32k (B)
long output, many steps, long context	78.9	100	89
long output, many steps, short context	64.9	99	70
short output, many steps, long context	18.6	58	33
long output, few steps, long context	13.6	50	39
short output, many steps, short context	18.1	23	40
long output, few steps, short context	11.6	15	25
short output, few steps, long context	4.6	15	21
short output, few steps, short context	3.8	9	18

Table 5: Failure rate by task class at 0.1 agent tasks/s, task-level FIFO, best N. Full breakdown: Sections G.7 and G.9.

B.5 Long agent tasks bear most of the failures

Every one of these classes fits in the budget alone at 16k and 32k, yet long-output, many-step tasks almost never finish (Table 5). They stay in the system longest, so they are the most likely to be running when an abort occurs. Those two classes are 28% of agent tasks but about 71% of the output demand. **The headline goodput of the tuned baselines is therefore partly achieved by losing the longest work**, and their latency percentiles, computed over completed tasks only, are flattered for the same reason. This matters for Phase 2: a policy could raise goodput by starving long tasks even further, so per-class results must be reported alongside it.

B.6 Retained context matters; offloading showed no clear benefit

Turning caching off cuts goodput 4.8× for task-level FIFO and 5.9× for request-level, and raises median latency about 3× (Table 6). Agent workloads depend on prefix reuse: 71–86% of prompt tokens are served from cache with caching on. Any Phase 2 policy must preserve this reuse.

We observed no clear benefit from offload under the tested conditions. At every rate tested, offload's mean goodput and cache reuse were close to resident's, with 95% confidence intervals that overlap widely; we did not run a formal test for a difference. Offload also used more host RAM at lower rates. One explanation is consistent with Section B.2: at an abort no idle context remains to move out of the way, so moving idle context out earlier would not address the cause.

This is not strong enough to rule offload out:

- Each comparison rests on five seeds with wide intervals, so a modest effect could go undetected.
- Every run used a single tool-delay setting (1 s median). Offload is about context that sits idle during tool delays, so longer or more variable delays could change the result.

The tool-delay sweep listed in Section E is the test that could overturn this conclusion.

B.7 Physical memory, overhead and cold start

- **VRAM follows the budget, not the load.** Peak VRAM is about 3.4, 4.3 and 6.1 GiB at 8k, 16k and 32k and varies little across rates, consistent with a pool reserved at launch. The slope is about 113 KB per token.
- **Host RAM follows load and caching.** It rises with load toward the `-cache-ram` limit (8192 MiB plus about 750 MiB for the process), and stays at 746 MiB with caching off. Retained context has a real memory cost outside the KV budget.
- **Policy overhead is negligible:** about 20–30 µs per decision, against model requests that take seconds.
- **Cold start is small.** Launch to healthy took a median of 2.8 s (A) and 1.6 s (B), and a first probe request took 0.75 s against 0.58 s warm (A), and 0.52 s against 0.53 s (B) (Section G.10). Steady-state results do not depend on it, as warm-up is excluded.

Baseline	Budget	N	Rate/s	Cache policy	Mach.	Goodput/min	Cached %	p50 s	Peak host RAM MiB
Task-level	16k	4	0.1	resident	A	2.73 ± 0.53	77 ± 3	20 ± 4	8507 ± 1057
Task-level	16k	4	0.1	off	A	0.57 ± 0.45	0	69 ± 27	746 ± 1
Request-level	16k	4	0.1	resident	A	1.95 ± 1.26	71 ± 4	27 ± 21	8767 ± 439
Request-level	16k	4	0.1	off	A	0.33 ± 0.32	0	67 ± 22	746 ± 2
Task-level	16k	4	0.025	resident	B	1.12 ± 0.27	85 ± 3	17 ± 7	4372 ± 1112
Task-level	16k	4	0.025	offload	B	1.18 ± 0.40	85 ± 2	16 ± 5	5091 ± 1282
Task-level	16k	4	0.05	resident	B	2.20 ± 0.65	83 ± 2	19 ± 4	6134 ± 1628
Task-level	16k	4	0.05	offload	B	2.20 ± 0.65	83 ± 2	18 ± 4	7789 ± 1244
Task-level	32k	16	0.05	resident	B	2.42 ± 0.66	86 ± 3	31 ± 11	8260 ± 1085
Task-level	32k	16	0.05	offload	B	2.73 ± 0.90	86 ± 2	26 ± 8	8709 ± 799
Task-level	32k	16	0.1	resident	B	3.12 ± 1.18	80 ± 3	27 ± 5	8940 ± 138
Task-level	32k	16	0.1	offload	B	2.98 ± 1.05	79 ± 3	27 ± 7	9016 ± 24

Table 6: Effect of cache policy. Each comparison stays on one machine. VRAM per row: Section G.3.

Candidate	Evidence	Verdict
Tune fixed N	Best N still fails about half of tasks at 0.1/s (Section B.1)	Keep as the baseline to beat
Increase the budget	Removes aborts but goodput can fall (Section B.3)	Not a policy; it costs memory
Offload idle context	No clear benefit at the one tool delay tested (Section B.6)	Not pursued now; revisit after a tool-delay sweep
Choose which idle context to evict	Aborts occur with nothing idle; deletions are rare at the tested tool delay (Section B.2)	Low leverage under tested conditions; possible later extension
Admit by footprint	Over-budget footprint coincided with 145 of 146 aborts (partly by construction); mean use is only 21–52% of budget	**Selected**

Table 7: Phase 2 candidates and the evidence for each.

C The dominant bottleneck

The bottleneck depends on load:

- **Below about 0.1 agent tasks/s, it is the combined KV footprint of concurrently running requests,** interacting with a runtime that can only fail all of them when it overflows. Failures here are pool exhaustion, output is well below the most observed at higher load, and mean memory use is well below the budget.
- **At about 0.1 agent tasks/s, compute pressure probably also matters.** Removing aborts by adding memory converted failures into timeouts rather than completions (Section B.3). This is an inference from offered load and failure causes, not a measured capacity limit.

Static tuning cannot resolve this, because a slot count is the wrong unit: four short tasks and four long tasks at the same N differ in footprint by an order of magnitude. Tuned N therefore has to trade queueing under small tasks against overflow under large ones.

D Phase 2 direction: footprint-based admission

Mechanism. Before admitting a model request, the proxy compares its known peak footprint (prompt tokens plus `n_predict`), added to the footprint of everything in flight, with the memory budget. If it does not fit, the request waits.

Why this one, from the measurements. Table 7 compares the candidates.

The signal is available and consistent with the aborts. Footprint is known exactly before admission and costs almost nothing to compute. Across the configurations in Table 3 where aborts occurred, 145 of 146 aborts in the measured window happened during a spell of over-budget projected footprint (Section G.5). About 72% of those spells ended in an abort (151 of 211). The rest ended safely, because requests finished before reaching their full projected size, which suggests the rule will sometimes hold back work unnecessarily. Measuring that cost is part of Phase 2. This coincidence is partly built into the measurement. Each model request counts at its full projected size from the moment it is admitted, so an overflow of the real pool is likely to fall inside an over-budget projection whether or not the projection is a useful signal. The 145 of 146 therefore shows that the signal is consistent with the aborts, not that holding requests back would prevent them; that is what H1 tests.

Hypotheses, each of which can fail.

- **H1.** At 0.025–0.05 agent tasks/s, footprint-based admission eliminates almost all overflow aborts and raises goodput over the best tuned N.

- **H2.** At 0.1 agent tasks/s the gain is smaller, if the compute pressure suggested in Section B.3 limits it. A sustained capacity measurement, run first, tests whether that limit exists. Little or no gain at this rate would be a valid result.
- **H3.** Without care, waiting for room will starve long agent tasks. The policy may need to favour tasks already in progress, since their work is invested. We will test this per task class.

Evaluation plan. Compare against FIFO and the best fixed N for each baseline, selected on development workloads only, using evaluation workloads with separate seeds and the held-out mix. Include varying arrival rates, bursty arrivals, and both budgets. Report goodput, latency percentiles, peak memory and failure rate per cause and per task class, as well as policy overhead. Compare against a fixed cache policy, and include a variant with the mechanism removed. Before any policy runs, measure sustained capacity: run a closed loop that keeps the server at fixed concurrency with no arrivals, warm-up or drain in the measured window, and record steady-state output tokens/s at each concurrency. This replaces the arrival-window estimate in Section B.3. A new pod must pass the same equivalence check against machine B first.

E Limitations

- **Idle periods were not varied.** Tool delay stayed at a 1 s median, so this phase does not determine how idle time affects memory pressure beyond what Section B.2 shows. A tool-delay sweep is still needed. It could also change the conclusions about offload (Section B.6) and about evicting idle context (Section D), because both depend on how long context sits idle.
- **No sustained capacity measurement.** The suspected compute limit in Sections B.3 and C is an inference, not a measurement. The output rates reported include output generated during the drain, divided by the arrival window, so they are not steady-state throughput.
- **Projected, not actual, occupancy.** The server does not report KV occupancy, so footprints are the proxy's calculations of prompt plus `n_predict`. Actual occupancy at an abort is lower than the projection, and the proxy cannot see the server's deletions of idle slots.
- **Small samples per run.** Each run has about 11–51 measured agent tasks, abort failures are correlated, and confidence intervals are correspondingly wide. Many differences between neighbouring settings are not significant.
- **Synthetic workload.** Parameters are checked against public trajectories, but tool-delay values are assumed, and token counts in that check use a stand-in tokenizer. The mix is uniform by design, and real workloads may have fewer very long tasks.
- **Two machines.** Machine B is about 4% slower per task, and SLOs differ per machine. Cross-machine points are marked, and the equivalence check covers one configuration only.
- **Single model and scope limits.** Results are for one model and precision on one GPU type. Only development workloads and Poisson arrivals were used; the 8k budget was run only at one rate; N above 16 was not tested, since at $N = 16$ concurrency peaked at 10–13. Rate calibration used one seed and is indicative only. The optional AWS CPU check was not done.

F Reproducibility

Versions are pinned in `configs/pins.toml`. Every sweep run records its full configuration, seed, versions, model hash, launch arguments and raw logs.

F.1 Evidence: the raw-result archives

The raw results are **not in Git**, because `results/` is ignored. A repository checkout alone therefore does not contain the evidence for this report. The two archives in Table 8 are the source of truth for every number above. They are attached, with their checksum file, to the repository's GitHub release `phase1-data`:

```
gh release download phase1-data -R zkan2002/CSE291A-Project -D phase1-archives
# then use phase1-archives as ARCHIVE_DIR
```

Archive	Machine	Size	Contents
`pod-results-2026-10-19.tar.gz`	A	24.7 MB	`results/phase1-dev/`: 102 complete sweep runs, plus 1 `.partial` run killed by the reboot. `results/calibrate-rates/`: 7 calibration runs. The sweep config as run, and pod logs.
`machine-b-final-2026-10-21.tar.gz`	B	17.2 MB	`results/phase1-dev-b/`: 100 complete sweep runs. The sweep config as run (generated grids included), and pod logs.

Table 8: Raw-result archives.

SHA-256 checksums are in `docs/reports/phase1-archives.sha256`. The sweep configs inside the archives are identical to `configs/phase1-dev.toml` and `configs/phase1-dev-b.toml` in the repository.

The analysis uses 200 of these sweep runs. Machine A's `.partial` run is excluded, and so are the two complete seeds of its configuration (request-level FIFO, $N = 16$, 32k, 0.1/s), because that configuration was re-run in full on machine B (ADR-0004).

F.2 Reproducing the analysis from the supplied logs

This needs no GPU. From a fresh checkout, with the two archives in `ARCHIVE_DIR` (for example, downloaded as in Section F.1):

```
uv sync
# verify the checksums and extract into results/phase1/
bash scripts/phase1-extract.sh ARCHIVE_DIR
# read results/phase1/ and write the tables and figures
bash scripts/phase1-analysis.sh
```

`phase1-extract.sh` refuses to overwrite a non-empty target. It produces:

```
results/phase1/machine-a/results/phase1-dev/runs/        machine A's sweep runs
results/phase1/machine-a/results/calibrate-rates/runs/   calibration runs
results/phase1/machine-b/results/phase1-dev-b/runs/      machine B's sweep runs
```

`phase1-analysis.sh` then runs these steps in order:

1. `agentserve report` for each machine and for calibration, into `results/phase1/report-a/`, `report-b/` and `report-calibration/`.
2. The equivalence check between the machines (prints `EQUIVALENT`).
3. `agentserve.best_n` on both reports. It prints the chosen N per baseline and budget, which must match the grids recorded in `configs/phase1-dev-b.toml`.
4. `scripts/phase1_analysis.py`, which writes the tables (`docs/reports/phase1-tables.md`), the appendix of this report (`docs/reports/phase1-appendix.tex`) and the figures (`docs/reports/phase1-figures/`).

Running this sequence from a fresh extraction of the archives reproduced the tables exactly. On a machine where `uv` is not on `PATH`, set it explicitly, for example `UV="py -3.11 -m uv" bash scripts/phase1-analysis.sh`.

F.3 Rerunning the experiments

This needs an NVIDIA GPU; we used an RTX 2000 Ada (16 GB) on RunPod. `docs/environment.md` covers pod setup. The sweep outputs go into the same layout as Section F.2, so the same analysis command reads them:

```
# build llama.cpp b11606, download the model, check both against pins.toml
bash scripts/runpod-setup.sh
uv run pytest && uv run pytest -m real_runtime
uv run agentserve sweep configs/calibrate-rates.toml \
    --out results/phase1/machine-a/results/calibrate-rates
uv run agentserve sweep configs/phase1-dev.toml \
    --out results/phase1/machine-a/results/phase1-dev
uv run agentserve sweep configs/phase1-dev-b.toml \
    --out results/phase1/machine-b/results/phase1-dev-b
bash scripts/phase1-analysis.sh
```

A few things differ from the original run:

- **Time:** each sweep takes roughly one to one and a half days of GPU time. Rerunning the same command resumes an interrupted sweep and skips runs that are already complete.
- **Best N is not recomputed by the sweeps:** the best-N grids are already in `configs/phase1-dev-b.toml`, so `best_n` in step 3 of the analysis only checks that the choice still holds.
- **One machine instead of two:** both configs run on the same machine, so `machine-a` and `machine-b` become directory names only. `configs/phase1-dev.toml` also completes the $N = 16$, 32k configurations that machine A never finished; the analysis then reads those from `machine-a`.
- **Results will vary:** task SLOs are re-measured on the new machine. A different GPU, driver or CPU quota changes absolute timings, and it can also change which failure mode dominates and which N is best. The reported confidence intervals describe variation across seeds on our two machines, not across hardware. On different hardware, compare the qualitative findings, and run the equivalence check (ADR-0004) before comparing any numbers directly.

- **Comparing against these results:** to compare a new machine with these results rather than replace them, first run the equivalence check against the supplied machine B runs (ADR-0004).

G Supplementary tables

These tables are generated from the raw runs by `scripts/phase1_analysis.py` (Section F.2); the same content is in `docs/reports/phase1-tables.md`. Values are means ± 95% CI half-widths across five seeds unless noted.

G.1 Tuning at 0.1 agent tasks/s, resident

Every concurrency setting tried for each baseline and memory budget. Latency covers completed agent tasks only.

Baseline	Budget	N	Mach.	Goodput/min	Throughput/min	Timeout %	Pool %	p50 s	p95 s	Queue s
Task-level FIFO	16k	1	A	0.28 ± 0.37	1.98 ± 0.30	69 ± 4	0 ± 0	359.2 ± 80.0	594.0 ± 86.2	50.1 ± 7.4
Task-level FIFO	16k	2	A	0.57 ± 0.30	2.62 ± 0.67	53 ± 6	6 ± 4	177.7 ± 29.2	331.6 ± 64.2	26.9 ± 4.2
Task-level FIFO	16k	4	A	2.73 ± 0.53	3.27 ± 0.82	1 ± 1	48 ± 9	20.2 ± 3.9	44.6 ± 18.9	1.0 ± 1.2
Task-level FIFO	16k	8	A	2.58 ± 0.61	2.58 ± 0.61	0 ± 0	60 ± 8	16.2 ± 4.7	40.3 ± 21.9	0.0 ± 0.0
Task-level FIFO	32k	1	A	0.28 ± 0.37	1.98 ± 0.30	69 ± 4	0 ± 0	359.5 ± 79.6	594.7 ± 85.2	50.1 ± 7.3
Task-level FIFO	32k	2	A	0.62 ± 0.35	2.75 ± 0.81	57 ± 7	0 ± 0	203.8 ± 49.7	364.4 ± 52.8	28.4 ± 4.6
Task-level FIFO	32k	4	A	1.12 ± 0.51	3.85 ± 0.91	40 ± 8	0 ± 0	153.9 ± 26.0	317.2 ± 40.6	15.6 ± 3.1
Task-level FIFO	32k	8	A	2.70 ± 0.94	4.15 ± 0.79	2 ± 5	33 ± 5	34.9 ± 7.2	137.2 ± 31.9	1.1 ± 1.8
Task-level FIFO	32k	16	B	3.12 ± 1.18	3.60 ± 1.11	0 ± 0	44 ± 12	26.5 ± 5.1	96.5 ± 53.2	0.0 ± 0.0
Request-level FIFO	16k	1	A	0.07 ± 0.21	1.32 ± 0.40	79 ± 6	0 ± 0	471.8 ± 288.6	756.5 ± 67.3	55.9 ± 8.5
Request-level FIFO	16k	2	A	0.25 ± 0.31	2.50 ± 1.29	53 ± 11	9 ± 7	228.7 ± 116.2	570.5 ± 84.4	38.4 ± 8.0
Request-level FIFO	16k	4	A	1.95 ± 1.26	3.08 ± 1.04	3 ± 8	50 ± 13	27.3 ± 20.8	92.7 ± 65.3	3.8 ± 5.6
Request-level FIFO	16k	8	A	2.58 ± 0.61	2.58 ± 0.61	0 ± 0	60 ± 8	16.8 ± 4.5	41.4 ± 21.1	0.0 ± 0.0
Request-level FIFO	16k	16	A	2.58 ± 0.61	2.58 ± 0.61	0 ± 0	60 ± 8	16.2 ± 4.8	40.4 ± 21.6	0.0 ± 0.0
Request-level FIFO	32k	1	A	0.07 ± 0.21	1.32 ± 0.45	79 ± 7	0 ± 0	493.6 ± 281.0	756.1 ± 66.6	55.7 ± 8.1
Request-level FIFO	32k	2	A	0.20 ± 0.26	2.98 ± 0.87	54 ± 8	0 ± 0	354.7 ± 197.9	637.3 ± 110.3	39.0 ± 7.3
Request-level FIFO	32k	4	A	0.53 ± 0.50	3.27 ± 1.68	47 ± 12	4 ± 11	205.7 ± 129.7	547.4 ± 82.5	28.4 ± 8.7
Request-level FIFO	32k	8	A	1.50 ± 0.92	3.02 ± 1.63	18 ± 28	36 ± 13	62.8 ± 41.9	204.9 ± 190.6	6.8 ± 9.1
Request-level FIFO	32k	16	B	3.08 ± 1.15	3.60 ± 1.11	0 ± 0	44 ± 12	26.6 ± 5.0	97.1 ± 54.5	0.0 ± 0.0

G.2 Arrival-rate ladder at best N

Baseline	Budget	N	Rate/s	Mach.	Goodput/min	SLO %	Failed %	p95 s	Cached %
Task-level FIFO	16k	4	0.025	B	1.12 ± 0.27	84 ± 19	16 ± 19	77.2 ± 13.3	85 ± 3
Task-level FIFO	16k	4	0.05	B	2.20 ± 0.65	70 ± 10	30 ± 10	87.9 ± 16.4	83 ± 2
Task-level FIFO	16k	4	0.1	B	2.95 ± 0.63	46 ± 8	48 ± 10	50.6 ± 40.9	76 ± 4
Request-level FIFO	16k	8	0.025	B	1.10 ± 0.28	82 ± 22	18 ± 22	75.3 ± 14.5	85 ± 3
Request-level FIFO	16k	8	0.05	B	2.12 ± 0.57	68 ± 11	32 ± 11	79.4 ± 29.2	82 ± 2
Request-level FIFO	16k	8	0.1	A	2.58 ± 0.61	40 ± 8	60 ± 8	41.4 ± 21.1	76 ± 5
Task-level FIFO	32k	16	0.025	B	1.40 ± 0.60	100 ± 0	0 ± 0	92.6 ± 26.9	87 ± 2
Task-level FIFO	32k	16	0.05	B	2.42 ± 0.66	78 ± 18	13 ± 22	134.3 ± 42.4	86 ± 3
Task-level FIFO	32k	16	0.1	B	3.12 ± 1.18	48 ± 14	44 ± 12	96.5 ± 53.2	80 ± 3
Request-level FIFO	32k	16	0.025	B	1.40 ± 0.60	100 ± 0	0 ± 0	92.5 ± 27.0	87 ± 2
Request-level FIFO	32k	16	0.05	B	2.42 ± 0.66	78 ± 18	13 ± 22	134.3 ± 42.4	86 ± 3
Request-level FIFO	32k	16	0.1	B	3.08 ± 1.15	47 ± 13	44 ± 12	97.1 ± 54.5	80 ± 3

G.3 Cache policies

Each comparison stays on one machine.

Baseline	Budget	N	Rate/s	Cache policy	Mach.	Goodput/min	Cached %	p50 s	Host RAM MiB	VRAM MiB
Task-level FIFO	16k	4	0.1	resident	A	2.73 ± 0.53	77 ± 3	20.2 ± 3.9	8507 ± 1057	4414 ± 62
Task-level FIFO	16k	4	0.1	off	A	0.57 ± 0.45	0 ± 0	69.0 ± 27.3	746 ± 1	4414 ± 62
Request-level FIFO	16k	4	0.1	resident	A	1.95 ± 1.26	71 ± 4	27.3 ± 20.8	8767 ± 439	4396 ± 61
Request-level FIFO	16k	4	0.1	off	A	0.33 ± 0.32	0 ± 0	67.0 ± 22.4	746 ± 2	4414 ± 62
Task-level FIFO	16k	4	0.025	resident	B	1.12 ± 0.27	85 ± 3	16.6 ± 6.5	4372 ± 1112	4390 ± 57
Task-level FIFO	16k	4	0.025	offload	B	1.18 ± 0.40	85 ± 2	15.9 ± 4.6	5091 ± 1282	4406 ± 60
Task-level FIFO	16k	4	0.05	resident	B	2.20 ± 0.65	83 ± 2	18.6 ± 3.6	6134 ± 1628	4410 ± 59
Task-level FIFO	16k	4	0.05	offload	B	2.20 ± 0.65	83 ± 2	17.5 ± 3.9	7789 ± 1244	4409 ± 64
Task-level FIFO	32k	16	0.05	resident	B	2.42 ± 0.66	86 ± 3	31.1 ± 11.2	8260 ± 1085	6220 ± 62
Task-level FIFO	32k	16	0.05	offload	B	2.73 ± 0.90	86 ± 2	25.7 ± 7.7	8709 ± 799	6201 ± 63
Task-level FIFO	32k	16	0.1	resident	B	3.12 ± 1.18	80 ± 3	26.5 ± 5.1	8940 ± 138	6184 ± 51
Task-level FIFO	32k	16	0.1	offload	B	2.98 ± 1.05	79 ± 3	27.0 ± 6.7	9016 ± 24	6202 ± 62

G.4 Overflow events (from raw logs)

Footprints are the proxy's projections (prompt plus n_predict), not the runtime's actual occupancy. Output figures include output generated during the drain, divided by the arrival window. Recorded output counts the tokens returned by model requests that completed. A request cut off by an abort returns nothing, so the tokens it had already generated are not counted.

Configuration	Mach.	Aborts/run	Failed requests/abort	In-flight at abort % of budget	Retained at abort, tokens	Mean admitted % of budget	Output tok per window-s	Recorded output on failed tasks %
Task-level, N=4, 16k, 0.025/s	B	0.8 ± 1.0	2.8	104	0	21	34 ± 11	21 ± 27
Task-level, N=4, 16k, 0.05/s	B	2.2 ± 1.0	3.5	106	0	35	59 ± 15	32 ± 14
Task-level, N=4, 16k, 0.1/s	A	6.4 ± 1.4	3.8	108	0	52	85 ± 26	63 ± 7
Request-level, N=8, 16k, 0.1/s	A	6.2 ± 1.0	4.9	117	0	42	68 ± 18	58 ± 7
Task-level, N=16, 32k, 0.05/s	B	0.4 ± 0.7	8.0	111	0	35	74 ± 7	15 ± 25
Task-level, N=16, 32k, 0.1/s	B	2.4 ± 0.7	9.3	111	0	43	89 ± 29	39 ± 20
Task-level, N=4, 32k, 0.1/s	A	0.0 ± 0.0	–	–	–	51	149 ± 24	2 ± 1
Request-level, N=16, 8k, 0.1/s	A	12.8 ± 2.2	2.9	118	63	35	44 ± 13	62 ± 9

G.5 Over-budget footprint spells and aborts

A spell is a stretch of time in which the projected in-flight footprint exceeds the budget. Counts are summed over all five seeds. Each model request counts at its full projected size from admission, so aborts are likely to fall inside a spell by construction; these counts show the signal is consistent with the aborts, not that holding requests back would prevent them.

Configuration	Mach.	Over-budget spells	Spells ending in abort	Aborts in window	Aborts without a spell
Task-level, N=4, 16k, 0.025/s	B	6	4	4	0
Task-level, N=4, 16k, 0.05/s	B	13	12	11	0
Task-level, N=4, 16k, 0.1/s	A	55	31	27	1
Request-level, N=8, 16k, 0.1/s	A	43	28	28	0
Task-level, N=16, 32k, 0.05/s	B	3	2	2	0
Task-level, N=16, 32k, 0.1/s	B	13	10	10	0
Task-level, N=4, 32k, 0.1/s	A	0	0	0	0
Request-level, N=16, 8k, 0.1/s	A	78	64	64	0

G.6 Output per arrival-window second at 0.1 agent tasks/s, resident

Highest six configurations. All output generated for the measured agent tasks, drain included, divided by the arrival window. This is not a steady-state capacity.

Configuration	Mach.	Output tokens per arrival-window s
Request-level, N=2, 32k, 0.1/s	A	152 ± 26
Request-level, N=4, 32k, 0.1/s	A	150 ± 39
Task-level, N=4, 32k, 0.1/s	A	149 ± 24
Request-level, N=2, 16k, 0.1/s	A	140 ± 32
Task-level, N=2, 32k, 0.1/s	A	136 ± 27
Request-level, N=1, 32k, 0.1/s	A	133 ± 19

G.7 By task class at best N, 0.1 agent tasks/s, resident

Means across five seeds. p50 covers completed agent tasks only (– where none completed).

Configuration	Task class	Mach.	Tasks/run	Failed %	SLO %	p50 s
Task-level, N=4, 16k	long_ctx-long_out-few_steps	A	6.6	50	50	20.1
Task-level, N=4, 16k	long_ctx-long_out-many_steps	A	6.0	100	0	–
Task-level, N=4, 16k	long_ctx-short_out-few_steps	A	6.8	15	52	9.8
Task-level, N=4, 16k	long_ctx-short_out-many_steps	A	5.6	58	42	26.4
Task-level, N=4, 16k	short_ctx-long_out-few_steps	A	8.4	15	68	24.0
Task-level, N=4, 16k	short_ctx-long_out-many_steps	A	8.6	99	1	106.5
Task-level, N=4, 16k	short_ctx-short_out-few_steps	A	3.6	9	73	6.2
Task-level, N=4, 16k	short_ctx-short_out-many_steps	A	5.6	23	75	23.9
Task-level, N=16, 32k	long_ctx-long_out-few_steps	B	6.6	39	56	35.7
Task-level, N=16, 32k	long_ctx-long_out-many_steps	B	6.0	89	11	125.2
Task-level, N=16, 32k	long_ctx-short_out-few_steps	B	6.8	21	69	8.1
Task-level, N=16, 32k	long_ctx-short_out-many_steps	B	5.6	33	67	33.6
Task-level, N=16, 32k	short_ctx-long_out-few_steps	B	8.4	25	51	28.5
Task-level, N=16, 32k	short_ctx-long_out-many_steps	B	8.6	70	30	111.3
Task-level, N=16, 32k	short_ctx-short_out-few_steps	B	3.6	18	52	7.6
Task-level, N=16, 32k	short_ctx-short_out-many_steps	B	5.6	40	60	30.3

G.8 8k overflow cliff by task class

Request-level FIFO, N=16, 8k, 0.1 agent tasks/s, machine A.

Task class	Tasks/run	Failed %	Fits budget alone
long_ctx-long_out-few_steps	6.6	81	yes
long_ctx-long_out-many_steps	6.0	100	no
long_ctx-short_out-few_steps	6.8	55	yes
long_ctx-short_out-many_steps	5.6	94	yes
short_ctx-long_out-few_steps	8.4	47	yes
short_ctx-long_out-many_steps	8.6	96	yes
short_ctx-short_out-few_steps	3.6	14	yes
short_ctx-short_out-many_steps	5.6	61	yes

G.9 Unloaded completion time by task class

Machine A, reference setup (resident, caching on, 32k), the agent tasks of all five seeds of one configuration's workload.

Task class	Tasks	Median s
long_ctx-long_out-few_steps	34	13.6
long_ctx-long_out-many_steps	31	78.9
long_ctx-short_out-few_steps	37	4.6
long_ctx-short_out-many_steps	31	18.6
short_ctx-long_out-few_steps	47	11.6
short_ctx-long_out-many_steps	48	64.9
short_ctx-short_out-few_steps	22	3.8
short_ctx-short_out-many_steps	29	18.1

G.10 Cold start

Medians over every server launch.

Machine	Launches	Launch to healthy s	First probe s	Warm probe s
A	102	2.79	0.75	0.58
B	100	1.62	0.52	0.53